설비 고장을 예측하는 AI 데이터 분석

Pandas 데이터 선택·필터링

조건 필터링과 정렬

posco

CONTENTS

목차

01 조건으로 행 거르기 — 참·거짓 목록과 표 전용 조건 도구

02 정렬 — 위험한 순서로 줄 세우기와 복사본 함정

03 종합 실습 — 조건과 정렬로 리포트 만들기

posco

이 단원의 실데이터 — 설비ID로 읽는 주조 라인

조건 필터링·정렬을 연습할 데이터가 어떤 설비인지 먼저 이해

설비ID 가 이 단원의 열쇠
DIE-A-01 형식이다. 앞의 DIE 는 다이캐스팅, 가운데 한 글자가 라인(A•B•C), 뒤 두 자리가 호기다.
로그 200줄에 다섯 대가 섞여 있다 → A라인 두 대(63•48샷), B라인 두 대(36•38샷), C라인 한 대(15샷).

왜 이 구조가 중요한가
「A라인 전체를 보자」와 「A라인 1호기만 보자」는 다른 질문이다.
앞은 글자가 들어 있는지를 묻는 조건이고, 뒤는 정확히 일치하는지를 묻는 조건이다.
같은 열에 두 층의 정보가 들어 있기 때문에 조건 도구를 골라 쓰는 연습에 그대로 쓰인다.

빈 칸이 있는 열
실린더압력•주조압력•사이클타임•비스킷두께•형체력 다섯 열에 각각 14건씩 빈 칸이 있다.
빈 칸은 어떤 비교를 해도 거짓이 되므로 조건으로 거를 때 양쪽 어디에도 들어가지 않는다.

정상과 이상
비스킷두께가 지나치게 두껍거나 사이클타임이 길면 주의•불량으로 기록된다.
200샷 중 양품 162 • 주의 18 • 불량 20이다.

posco

01

조건으로 행 거르기

참·거짓 목록으로 행을 고르고, 표에만 있는 조건 도구를 더하기

posco

조건 거는 법은 배열과 똑같다

앞 단원에서 배열에 걸던 조건이 표에서도 그대로 통한다 (기호도 괄호 규칙도 동일)

비교 항목	NumPy 배열	Pandas 표
조건 만들기	`arr[:, 5] >= 16`	`df['비스킷두께'] >= 16`
만들어지는 것	참·거짓 배열	**참·거짓 목록 (Series)**
골라내기	`arr[조건]`	`df[조건]`
두 조건 묶기	`(A) & (B)` · `(A) | (B)`	`(A) & (B)` · `(A) | (B)`
표에만 있는 것	없음	`isin` · `between` · `str.contains`

posco

Boolean Series 코드

열 전체에 조건을 걸면 행마다 참•거짓이 담긴 목록(Series)이 생긴다 — 배열에 걸던 것과 같은 일

CODE

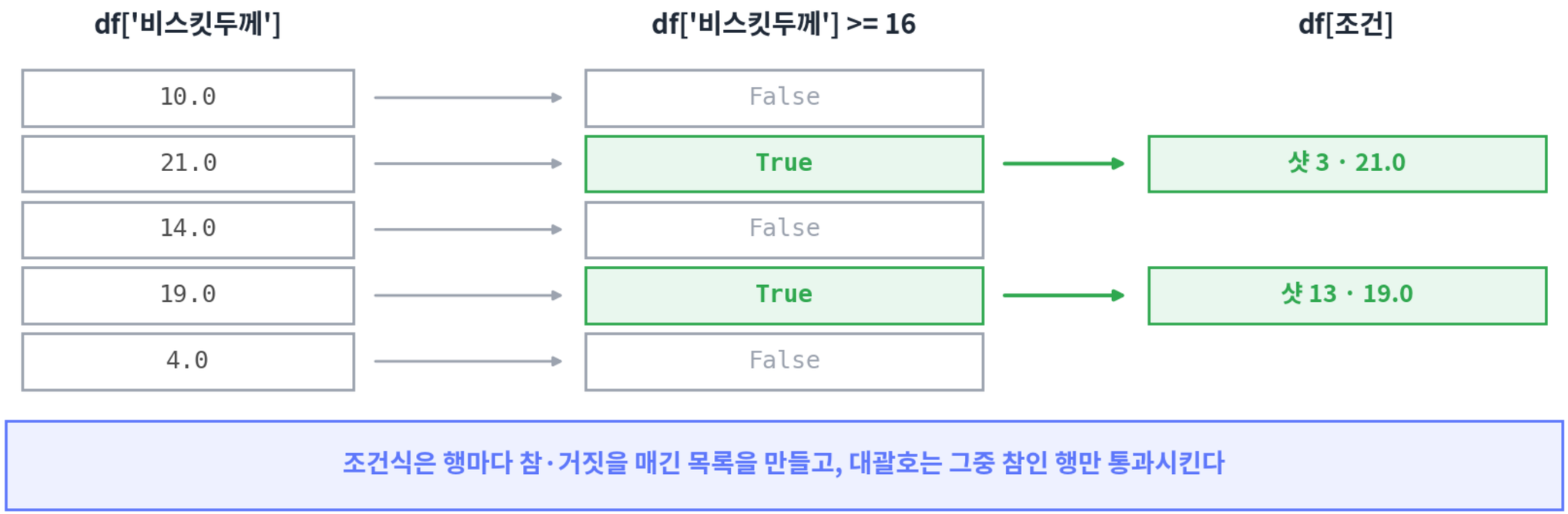
```
조건 = df['비스킷두께'] >= 16
print(조건.head(3).tolist())
# [False, False, False]

print(조건.sum())      # 74  (참의 개수)
print(len(조건))       # 200 (전체 행 수)
```

sum은 참을 1로 세므로 조건에 맞는 행이 몇 개인지 바로 확인 가능

조건은 참·거짓 목록을 만들고 대괄호가 걸러 낸다

조건식이 곧 행을 고르는 것이 아니다 — 두 단계를 나눠 보면 헷갈릴 일이 없다



조건식은 행마다 참·거짓을 매길 뿐이고, 행을 줄이는 것은 대괄호다.

그래서 조건만 만들면 행 수가 그대로고, 대괄호에 넣어야 줄어든다

posco

빈 칸은 조건에서 어느 쪽에도 들어가지 않는다

정반대 조건 둘을 더해도 전체가 되지 않는 이유 —실무 데이터에서 반드시 만나는 함정



빈 칸은 크지도 작지도 않다. 어떤 비교를 해도 거짓이 된다.

posco

실습 1. 한 조건으로 행 거르기

조건 하나를 만들어 참의 개수를 세고, 그 조건으로 행을 골라내기

목표

비스킷두께가 기준을 넘는 샷을 골라내고, 정반대 조건과 개수를 더해 확인

단계

① 200줄짜리 다이캐스팅 로그를 읽어 전체 크기를 확인

② 비스킷두께가 16 이상이라는 조건을 만들어 참의 개수를 출력

③ 그 조건으로 True 갯수를 출력

④ 16 미만 조건도 만들어 두 개수를 더한 뒤 전체와 비교

예상 결과

조건에 맞는 줄 74 · 반대 조건 112 · 둘을 더해도 186으로 200에 못 미침

실습 1 결과 해석

숫자 세 개만 맞으면 이 실습은 끝난다

원리

조건은 지도, 대괄호가 체

조건식은 행마다 참·거짓을 매긴 목록을 만들 뿐이다.

행이 실제로 줄어드는 것은 그 목록을 대괄호에 넣는 순간이다.

함정

정반대 조건을 더해도 전체가 안 된다

빈 칸이 있는 행은 어떤 비교를 해도 거짓이라 양쪽 어디에도 들어가지 않는다.

74 + 112 = 186 이고 나머지 14가 빈 칸이다.

검산

74 • 112 • 200

조건에 맞는 줄이 74,
반대 조건이 112,
전체가 200이면 맞다.

합이 200으로 딱 떨어졌다면 다른 파일을 읽은 것이다.

조건을 걸었을 때는 골라진 줄 수와 전체 줄 수를 같이 찍어 보는 습관을 들일 것

posco

실습 1 정답

조건을 만들고, 개수를 세고, 골라내는 세 단계

CODE

```
import pandas as pd

df = pd.read_csv('data/13_diecasting_shot.csv')
print(len(df))                    # 200

조건 = df['비스킷두께'] >= 16
print(조건.sum())                 # 74

두꺼움 = df[조건]
print(len(두꺼움))                # 74

얇음 = df[df['비스킷두께'] < 16]
print(len(얇음))                  # 112
print(74 + 112)                   # 186  ← 200이 아니다
```

마지막 두 줄이 이 실습의 결론 — 빈 칸 14개가 양쪽 어디에도 없다

posco

조건 2개 이상 — & 와 | 그리고 괄호

앞 단원 배열에서 쓰던 그대로 — 다만 and·or 로 묶으면 오류

두 조건 모두 만족은 & , 하나라도 만족은 |

and·or 는 값 하나끼리 묶는 도구 — 행마다 참·거짓이 든 목록에는 못 씀

각 조건은 반드시 괄호로 — 빼면 & 가 비교보다 먼저 계산되려 해서 오류

```
(df['비스킷두께']>=16) & (df['사이클타임']>=25)          정상
df['비스킷두께']>=16 & df['사이클타임']>=25          오류
```

posco

복합 조건 코드

각 조건을 괄호로 감싸 & 또는 |로 연결

CODE

```
both = df[(df['비스킷두께']>=16) & (df['사이클타임']>=25)]
either = df[(df['비스킷두께']>=16) | (df['사이클타임']>=25)]

print(len(both), len(either))   # 72 86
```

둘 다 만족은 72행, 하나라도 만족은 86행 — 그리고가 항상 더 적다

posco

부정 조건 ~

물결 기호 ~를 조건 앞에 붙이면 참과 거짓이 뒤집힘

물결 기호를 조건 앞에 붙이면 **참과 거짓이 뒤집힘**

`df[~(df['품질등급'] == '불량')]` 는 불량이 아닌 행 – 180행

조건이 길어질수록 통째로 뒤집는 쪽이 짧고 읽기 쉬움

> 파이썬 기초의 **not**과 같은 역할이지만, Boolean Series에는 **~**를 사용
>
> 키보드에서 보통 숫자 1 왼쪽에 위치

posco

실습 2. 두 조건 묶기

그리고와 또는으로 조건을 묶고, 통째로 뒤집는 것까지 해 보기

목표

비스킷두께와 사이클타임 두 조건을 각각 그리고·또는로 묶어 개수를 비교

단계

① 200줄짜리 다이캐스팅 로그를 읽어 변수에 담기

② 비스킷두께 조건과 사이클타임 조건을 각각 괄호로 감싸 그리고로 묶기

③ 같은 두 조건을 또는으로 묶어 줄 수를 비교

④ 품질등급이 불량인 조건을 만들고 물결 기호로 통째로 뒤집기

예상 결과

그리고는 72줄 · 또는은 86줄 · 불량이 아닌 행은 180줄

posco

실습 2 결과 해석

묶는 기호와 괄호, 그리고 개수의 방향을 확인

원리

여러 행짜리는 and 가 아니라 &

and 와 or 는 참·거짓 값 하나끼리 묶는 도구다.

행마다 참·거짓이 잔뜩 든 목록에는 & 와 | 를 쓴다.

함정

각 조건을 괄호로 안 감쌈

괄호를 빼면 & 가 비교보다 먼저 계산되려 해서 오류가 난다.

조건이 하나라도 둘 이상이면 무조건 괄호부터 친다고 외울 것.

검산

72 • 86 • 180

그리고가 72, 또는이 86,
뒤집은 결과가 180이면 맞다.

그리고가 또는보다 많게 나왔다면 기호를 바꿔 쓴 것이다.

posco

실습 2 정답

괄호로 감싸 & 와 | 로 묶고, ~ 로 뒤집기

CODE

```
df = pd.read_csv('data/13_diecasting_shot.csv')

두꺼움 = df['비스킷두께'] >= 16
느림   = df['사이클타임'] >= 25

print(len(df[두꺼움 & 느림]))     # 72   둘 다 만족
print(len(df[두꺼움 | 느림]))     # 86   하나라도 만족

# 조건을 변수에 담지 않고 한 줄로 쓸 때는 괄호 필수
print(len(df[(df['비스킷두께']>=16) & (df['사이클타임']>=25)]))  # 72

print(len(df[~(df['품질등급'] == '불량')]))   # 180
```

조건을 변수에 담아 두면 괄호를 덜 써도 되고 코드도 훨씬 읽기 쉬워진다

posco

isin으로 여러 값 매칭

값이 주어진 목록 안에 있는지 한 번에 검사

isin은 **값이 목록 안에 있는지**를 한 번에 검사

`df['품질등급'].isin(['주의', '불량'])` 처럼 목록으로 전달

세로 막대를 여러 번 쓰는 것보다 훨씬 깔끔

> 주의거나 불량인 행을 찾을 때 — 세로 막대로 두 번 잇는 대신 목록 하나로
>
> 품질등급이 그중 하나면 참이 되는 참·거짓 목록 생성 —여기서는 38행이 참

between으로 범위 조건

값이 두 경계 사이에 있는지 한 번에 검사 — 양쪽 경계 모두 포함

01 • 범위 검사

`df['비스킷두께'].between(10, 16)`

한 줄로 범위 검사

같은 열에 아래위 경계가 둘 다 있을 때
복합 조건보다 짧고 읽기 쉬움

02 • 양 끝 포함

between(10, 16) 은

10과 16도 포함

10 초과 16 미만으로 착각하면
경계에 걸린 샷이 빠져 결과가 미묘하게 달라짐

posco

문자열 조건 str.contains

글자 열에서 일부만 일치해도 골라내기 — 설비ID 로 라인 단위 조회

글자 열에서 특정 글자가 들어 있는지 검사

df['설비ID'].str.contains('A') 로 A라인 전체 — 111행

숫자 열에는 쓸 수 없음 — 샷 열에 쓰면 AttributeError

> 설비ID 는 DIE-A-01 형식이라 == 로는 호기 하나만, contains 로는 라인 전체를 고름
>
> 여기서 str 은 이 열을 글자로 다루겠다는 신호

posco

문자열 나누고 다듬기 — split · replace · strip

설비ID 를 라인과 호기로 쪼개면 라인 단위 분석이 한 줄로 끝난다

CODE

```
print(df['설비ID'].head(1).tolist())               # ['DIE-A-01']
print(df['설비ID'].str.split('-').head(1).tolist())  # [['DIE', 'A', '01']]

라인 = df['설비ID'].str.split('-').str[1]     # 가운데 조각만
호기 = df['설비ID'].str.split('-').str[2]     # 마지막 조각만
print((라인=='A').sum(), (라인=='B').sum(), (라인=='C').sum())   # 111 74 15
print((호기=='01').sum(), (호기=='02').sum())                    # 114 86

print(df['설비ID'].str.replace('DIE-', '', regex=False).head(1).tolist())
# ['A-01']      앞머리를 걷어내 짧은 이름으로
```

contains 는 「들어 있나」만 답하고, split 은 실제로 라인이라는 새 값을 만들어 준다 · 앞뒤 공백은 str.strip()

posco

조건 도구 한눈에 정리

자유롭게 조합하면 아무리 복잡한 조건도 표현 가능

도구	용도	예시
`&`	**모두 만족**	`(A) & (B)`
`|`	**하나라도 만족**	`(A) | (B)`
`isin`	**목록 매칭**	`.isin(목록)`
`between`	**범위 검사**	`.between(소, 대)`
`~`	**부정·뒤집기**	`~(조건) · ~isin · ~between`

posco

실습 3. 목록과 범위로 거르기

표에만 있는 세 도구를 한 번씩 써 보고, 같다와 포함의 차이를 확인

목표

목록으로 고르기, 범위로 고르기, 글자 일부로 고르기를 각각 적용

단계

① 200줄짜리 다이캐스팅 로그를 읽어 변수에 담기

② 품질등급이 주의거나 불량인 샷을 목록 매칭으로 골라 줄 수를 출력

③ 비스킷두께가 10과 16 사이인 샷을 범위 검사로 골라 줄 수를 출력

④ 설비 이름에 A가 들어간 샷과 A라인 1호기만 각각 골라 줄 수를 비교

예상 결과

목록 매칭 38줄 · 범위 검사 117줄 · 글자 포함 111줄 · 정확히 일치 63줄

posco

실습 3 정답

목록 •범위 •글자 포함 세 도구를 차례로 적용

CODE

```
df = pd.read_csv('data/13_diecasting_shot.csv')

목록 = df[df['품질등급'].isin(['주의', '불량'])]
print(len(목록))                                  # 38

범위 = df[df['비스킷두께'].between(10, 16)]   # 10과 16도 포함
print(len(범위))                                  # 117

A라인 = df[df['설비ID'].str.contains('A')]
print(len(A라인))                                 # 111

한호기 = df[df['설비ID'] == 'DIE-A-01']
print(len(한호기))                               # 63
```

마지막 두 줄이 포함과 일치의 차이 — 111과 63

02

정렬과 복사본

위험한 순서로 줄 세우기 · 걸러낸 결과를 고칠 때의 복사본 함정

sort_values 단일 키 정렬

> **sort_values 괄호에 기준 열 이름을 적으면 그 열로 정렬**

`sort_values` 괄호에 **기준 열 이름**을 적으면 그 열로 정렬

기본은 작은 값부터인 **오름차순**

한 열로 정렬해도 **모든 열이 행 단위로 함께 이동**

```
df.sort_values('비스킷두께')
```

비스킷두께 열로 줄 세움 —작은 값이 위로

정렬은 기준 열만 옮기는 것이 아니다

행이 통째로 따라 움직인다 — 그래서 정렬해도 각 행의 값들은 짝이 유지된다

정렬 전

샷	비스킷두께	품질등급
1	10.0	양품
3	21.0	주의
5	14.0	불량
13	19.0	주의

sort_values →

비스킷두께 내림차순

샷	비스킷두께	품질등급
3	21.0	주의
13	19.0	주의
5	14.0	불량
1	10.0	양품

기준 열만 움직이는 것이 아니라 행이 통째로 따라간다 — 샷 3의 품질등급은 정렬 뒤에도 그대로 주의

기준 열을 옮기면 같은 행의 나머지 값도 함께 따라간다.

정렬은 원본을 바꾸지 않는다 — 결과를 변수에 담지 않으면 아무 일도 일어나지 않음

posco

ascending과 다중 키 정렬

내림차순과 우선순위가 있는 다중 정렬

`ascending=False`를 주면 큰 값부터인 **내림차순**으로 정렬

기준 열을 **리스트로 묶으면** 우선순위가 있는 다중 정렬

```
df.sort_values('사이클타임', ascending=False)
```

사이클타임 큰 값부터 — 지연이 심한 샷을 위로

```
df.sort_values(['품질등급', '사이클타임'], ascending=[True, False])
```

먼저 품질등급으로, 품질등급이 같으면 사이클타임으로 —우선순위 정렬

정렬 후 상위 N개 추출

정렬과 상위 추출을 점으로 잇는 메서드 체이닝

정렬 후 **head**를 붙이면 상위 몇 개만 추출

정렬과 상위 추출을 **점으로 이어 한 줄**로 작성 가능

상위 몇 개만 필요할 때는 nlargest 로 한 번에 — 정렬 결과와 같음

```
df.sort_values('비스킷두께', ascending=False).head(5)   ==   df.nlargest(5, '비스킷두께')
```

posco

실습 4. 위험한 순서로 줄 세우기

기준 하나로 줄을 세우고, 기준 둘로 우선순위를 준 정렬까지 해 보기

목표

비스킷두께가 큰 순서로 줄을 세우고 상위 몇 줄만 뽑아 확인

단계

① 200줄짜리 다이캐스팅 로그를 읽어 변수에 담기

② 비스킷두께가 큰 순서로 줄을 세워 상위 5줄만 출력

③ 품질등급을 먼저, 같으면 사이클타임이 큰 순서로 세워 상위 5줄을 출력

④ 정렬한 뒤 원본의 앞 5줄이 그대로인지 확인

예상 결과

상위 5줄의 샷 번호는 198 192 43 171 197 • 원본은 그대로 유지

실습 4 결과 해석

정렬의 세 가지 성질을 확인

원리

행이 통째로 따라 움직인다

기준 열만 정렬되는 것이 아니라
같은 행의 값들이 함께 이동한다.

그래서 정렬한 표에서도
각 줄은 여전히 한 샷의 기록이다.

함정

정렬 결과를 변수에 안 담음

정렬은 원본을 바꾸지 않는다.
화면에 한 번 보이고 끝난다.

이어서 쓰려면 반드시 새 변수에 담아야 한다.

검산

샷 198이 맨 위

상위 5줄의 샷 번호가
198 192 43 171 197 이면 맞다.

작은 값이 위로 왔다면
내림차순 옵션을 빠뜨린 것이다.

정렬 뒤 맨 위 한 줄이 필요하면 이름표가 아니라 순서로 불러야 한다

posco

실습 4 정답

단일 기준 정렬과 우선순위가 있는 다중 정렬

CODE

```
df = pd.read_csv('data/13_diecasting_shot.csv')

위험순 = df.sort_values('비스킷두께', ascending=False)
print(위험순['샷'].head(5).tolist())
# [198, 192, 43, 171, 197]

우선순위 = df.sort_values(['품질등급', '사이클타임'],
                          ascending=[True, False])
print(우선순위['샷'].head(5).tolist())
# [200, 181, 182, 191, 196]   — 품질등급 불량이 먼저

print(df['샷'].head(3).tolist())   # [1, 2, 3]  원본은 그대로
```

다중 정렬의 방향은 열마다 따로 — 품질등급은 가나다순, 사이클타임은 큰 값부터

posco

필터링 결과 저장과 copy

걸러낸 결과를 따로 저장해 수정하려면 .copy()로 독립 복사본

걸러낸 결과를 따로 저장해 수정하려면 **copy**로 복사본 생성

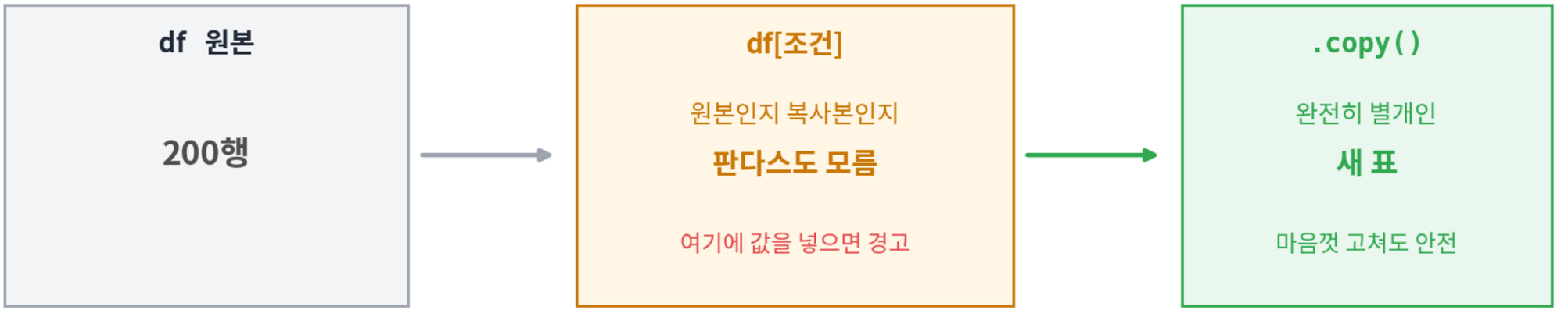
df[조건].copy()는 원본과 분리된 독립 복사본

복사본은 새 열 추가나 값 변경을 **경고 없이** 수행

> 불량 샷만 골라낸 뒤 위험등급 열을 새로 붙이는 식의 작업에 적합

걸러낸 결과를 고치려면 복사본이라고 못 박아야 한다

판다스도 그것이 원본의 일부인지 새 표인지 모른다 —그래서 경고가 뜬다



걸러낸 결과를 변수에 담아 고칠 생각이면 그 자리에서 .copy() 를 붙인다 — SettingWithCopyWarning 은 이 한 줄로 사라진다

걸러낸 결과를 변수에 담아 값을 고칠 생각이면 그 자리에서 복사본으로 만든다.

경고의 원인과 해결

.copy() 또는 .loc — 두 가지 해결 방법

01 · 원인

원본 일부인지
새 복사본인지 불확실

값을 바꾸면 원본까지 바뀔 수도,
복사본만 바뀔 수도 있어 예측 어려움

02 · 해결

따로 수정은 `.copy()`
원본 수정은 `.loc`

이 행의 이 열을 바꿔라라고
loc으로 명확히 지정

새 열 만들기 — 거른 결과에 판정 붙이기

없는 이름에 값을 넣으면 열이 새로 생긴다 — 분석 결과를 표에 남기는 법

CODE

```
불량 = df[df['품질등급'] == '불량'].copy()
불량['위험등급'] = '높음'           # 없는 이름에 대입 → 새 열
print(불량.shape)                  # (20, 9)    열이 8에서 9로

판정 = df.copy()
판정['라인'] = 판정['설비ID'].str.split('-').str[1]   # 앞에서 만든 값을 열로
판정.loc[판정['비스킷두께'] >= 16, '두께판정'] = '두꺼움'
print(판정['두께판정'].notna().sum())    # 74     조건에 맞는 행만 채워짐

판정['여유형체력'] = 판정['형체력'] - 250        # 열끼리·숫자와 계산도 가능
print(판정['여유형체력'].head(3).tolist())       # [8.0, 7.0, 4.0]
```

거른 결과에 열을 붙일 것이면 .copy() 를 먼저 — 조건에 안 맞는 행은 빈 칸으로 남는다

posco

03

종합 실습

조건으로 거르고 줄을 세워 한 장짜리 리포트로 만들기

posco

실습 5. 조건과 정렬로 리포트 만들기

거르고 줄 세우고 필요한 칸만 남겨 한 장짜리 점검표로 만들기

목표

양품이 아니면서 비스킷두께가 기준을 넘는 샷을 위험한 순서로 정리

단계

① 200줄짜리 다이캐스팅 로그를 읽어 변수에 담기

② 양품이 아니고 비스킷두께가 16 이상인 조건을 그리고로 묶어 걸러내기

③ 걸러낸 결과를 비스킷두께가 큰 순서로 줄 세우기

④ 설비 이름과 비스킷두께, 품질등급 세 칸만 남겨 상위 5줄을 출력

예상 결과

걸러진 샷 25줄 · 맨 위는 샷 198 · 비스킷두께 27.0 · 불량

실습 5 결과 해석

오늘 배운 것이 이 한 줄 안에 다 들어 있다

원리

거르고 • 세우고 • 추리는 순서

먼저 조건으로 행을 줄이고, 그다음 줄을 세우고, 마지막에 칸을 추린다.

이 순서가 가장 빠르고 읽기도 쉽다.

함정

빈 칸인 샷은 리포트에 안 올라온다

비스킷두께가 비어 있는 14개 샷은 조건에서 거짓이라 처음부터 빠진다.

그중에 진짜 문제가 있었을 수도 있다.

검산

25줄 • 샷 198

걸러진 결과가 25줄이고
맨 위가 샷 198이면 맞다.

더 많이 나왔다면 조건을 또는으로 묶은 것이다.

실습 5 정답

거르고 · 세우고 · 추리는 세 동작을 이어 붙이기

CODE

```
df = pd.read_csv('data/13_diecasting_shot.csv')

위험 = df[(df['품질등급'] != '양품') & (df['비스킷두께'] >= 16)]
print(len(위험))                    # 25

정렬 = 위험.sort_values('비스킷두께', ascending=False)
리포트 = 정렬[['설비ID', '비스킷두께', '품질등급']]
print(리포트.head(5))
#       설비ID  비스킷두께 품질등급
# 197  DIE-A-01   27.0   불량
# 191  DIE-B-01   26.0   불량
# 42   DIE-A-01   21.0   주의

# 세 동작을 점으로 이어 한 줄로도 가능 (메서드 체이닝)
```

왼쪽 번호는 원본에서의 이름표 — 정렬해도 따라온다는 앞 단원의 그 규칙